TrustPay Guardian: Fraud-Aware Digital Banking Platform

Product Requirements Document (PRD) v1.0 · 17 October 2026 · 7th-semester Major Project, B.Tech CSE (AI & ML) · Simulated (educational) digital banking platform with real-time fraud detection

> **Official project title (use it exactly in the README, UI header, Swagger title and report):** TrustPay Guardian: Fraud-Aware Digital Banking Platform. Short name: **TrustPay Guardian**. Repository: `trustpay-guardian`. Java base package: `com.trustpay.guardian`. Currency: INR (₹).

0. Rules for the AI coding agent (read first)

1. Build exactly what this PRD says. If something is unclear, choose the simplest option that matches the Java Full Stack syllabus (section 17) and log the decision in `docs/DECISIONS.md`.
2. Never guess dependency names, versions or APIs. Check official docs, Maven Central or npm before adding anything. Spring Boot 4 renamed several starters, so verify every one.
3. This is a simulated bank for learning: no real money, no real payment gateway, no real personal data. All seed data is fake.
4. The owner must be able to explain every line. Use package-by-feature, small classes, meaningful names, no Lombok (plain Java; records for DTOs) and short comments that explain why, not what.
5. Never commit secrets. Use environment variables and `.env.example`.
6. Work phase by phase (see the Prompt Guide). After each phase, run the build and tests and show proof before moving on.
7. Priorities: **[M]** must have, **[S]** should have, **[C]** could have. Finish all [M] items before any [S] item.
8. Money is always `BigDecimal` in Java and `DECIMAL(19,2)` in MySQL. Never use float or double for money.

1. Product overview

Problem. Digital banking apps must stop fraud without blocking genuine customers, and must protect every endpoint with strong authentication and role-based access.

Solution. TrustPay Guardian is a full-stack banking platform where customers manage accounts and transfer money. Every transfer is scored in real time by a transparent, rule-based fraud engine. Risky transfers are held or blocked and sent to a Fraud Analyst, while an Admin controls users, rules and settings.

Goals. (1) Secure JWT + RBAC authentication. (2) Correct, atomic money movement. (3) Explainable fraud scoring. (4) Polished, responsive UI with dark/light theme. (5) Tested, containerised, documented, GitHub-ready code.

Non-goals. Real payments or bank integrations, KYC document upload, mobile apps, machine-learning models (listed as a future extension), microservices (the app is a modular monolith).

2. Roles and permissions

Capability	Guest	CUSTOMER	FRAUD_ANALYST	ADMIN
View home page, register, log in	Yes	Yes	Yes	Yes
Own accounts, beneficiaries, transfers, statements	No	Yes	No	No
View fraud alert queue and details	No	No	Yes	Read-only
Release or confirm-block a flagged transaction	No	No	Yes	No
Manage users, roles, account freezes	No	No	No	Yes
Configure fraud rules and system settings	No	No	No	Yes
View audit logs and platform statistics	No	No	No	Yes
Own profile, password, theme, notifications	No	Yes	Yes	Yes

Public registration creates **CUSTOMER only**. The API must never accept a role from the client. The first ADMIN and a demo FRAUD_ANALYST are created by the dev seeder. Later analysts and admins are created by an ADMIN.

3. Technology stack

Layer	Choice	Notes
Language	Java 25 (LTS)	Syllabus covers Java 8 to 21; Java 25 is a superset, so everything taught still applies (records, streams, Optional, java.time, pattern matching)
Framework	Spring Boot 4.1.x (latest 4.1 patch)	Latest stable line as of Oct 2026. Spring Boot 3.5 lost free support in June 2026. Verify starter names (for example the modular `spring-boot-starter-webmvc` and `spring-boot-starter-flyway`)
Build	Maven with wrapper	Syllabus: Maven
Web/API	Spring Web MVC, Bean Validation, springdoc-openapi	Use the springdoc line compatible with Boot 4. Swagger UI at `/swagger-ui.html`
Persistence	Spring Data JPA (Hibernate), MySQL 9.7 LTS	`ddl-auto=validate`; schema owned by Flyway
Migrations	Flyway	Beyond syllabus; versioned SQL files
Security	Spring Security + JWT (JJWT, or Nimbus JOSE if JJWT conflicts with Boot 4)	Custom `OncePerRequestFilter` so the flow is explainable
Rate limiting	Bucket4j	Beyond syllabus; login and transfer endpoints
Logging/monitoring	SLF4J + Logback, Spring Boot Actuator	No PII or secrets in logs
Email (dev)	Spring Mail + MailHog container	[S]
Backend tests	JUnit 5, Mockito, MockMvc, `@SpringBootTest`, Testcontainers (MySQL), JaCoCo, SonarQube	
Frontend	React 19.x + TypeScript (latest stable), Vite (latest stable)	Node.js 24 LTS or the newest Active LTS

Layer	Choice	Notes
UI	Bootstrap 5.3 (`data-bs-theme`) + custom CSS variables	Syllabus: Bootstrap 5, HTML5, CSS3
Routing/state	React Router v7 (declarative mode; same concepts as the syllabus's v6), Context API (auth, theme), Redux Toolkit (notifications, alerts)	Syllabus covers both
Forms/HTTP	React Hook Form + Zod, Axios	
Charts/icons	Recharts, Bootstrap Icons	
Frontend tests	Vitest + React Testing Library (Jest-compatible API), one Playwright smoke test	Syllabus: Jest, RTL, Playwright awareness
DevOps	Docker, Docker Compose, Jenkinsfile, GitHub Actions	Syllabus: Docker, Jenkins

Version policy. Targets in the table above were checked against current release pages on 17 October 2026 for Spring Boot (4.1 is the latest stable line) and MySQL (9.7 is the latest LTS). Everything else (Java 25 LTS, Node.js, React, Vite, TypeScript, Bootstrap 5.3.x, Maven, Testcontainers, springdoc, JJWT, Flyway, Bucket4j, MySQL Connector/J and all npm packages) must be installed at its **latest stable release** and confirmed at install time. If the newest version of a library does not work with Spring Boot 4.1 or Java 25, pin the newest compatible version and record why in `docs/DECISIONS.md`.

4. Functional requirements

4.1 Authentication and session

- **FR-A1 [M] Register.** Fields: fullName (2–80), email (unique, stored lower-case), phone (10-digit Indian mobile), password (min 10 chars with upper, lower, digit, special), confirmPassword. Creates a CUSTOMER (status ACTIVE) and one default SAVINGS account funded with the `DEMO_OPENING_BALANCE` setting.
- **FR-A2 [M] Login** by email and password. Returns the access JWT in the JSON body and a refresh token in an `HttpOnly` , `SameSite=Strict` cookie (`Secure` when `COOKIE_SECURE=true`).
- **FR-A3 [M] Lockout.** After 5 consecutive failures set `locked_until = now + 15 min` . Every failure returns the same generic message (`Invalid credentials or account locked`). A success resets the counter.

- **FR-A4 [M] Refresh with rotation.** Each refresh issues a new refresh token and revokes the old one. Reuse of a revoked token revokes all refresh tokens of that user.
- **FR-A5 [M] Logout** revokes the refresh token and clears the cookie.
- **FR-A6 [M]** `GET /auth/me` returns the profile and role.
- **FR-A7 [S] Change password** (needs the current password; revokes all refresh tokens).
- **FR-A8 [C] Forgot password** through a one-time emailed token (MailHog in dev).

4.2 Accounts

- **FR-AC1 [M]** A customer sees only their own accounts. Lists show the account number masked except the last 4 digits.
- **FR-AC2 [M]** Open an extra account (SAVINGS or CURRENT), max 3 per customer. Account number = `TPG` + 10 random digits, unique.
- **FR-AC3 [M]** Statement with date range, pagination (10/20/50) and newest-first order. **[S]** CSV export.
- **FR-AC4 [M]** Available balance = balance minus the sum of the account's outgoing `PENDING_REVIEW` transfers.
- **FR-AC5 [M]** A FROZEN account cannot send or receive. The UI shows a banner.

4.3 Beneficiaries

- **FR-B1 [M]** Add a beneficiary by TrustPay account number plus nickname. The account must exist, must not be the user's own, and must be unique per user. Show the holder's masked name for confirmation.
- **FR-B2 [M]** List and delete. `created_at` drives fraud rule R4.

4.4 Money movement

- **FR-T1 [M] Transfer** from the customer's own account to a saved beneficiary. Min ₹1, max `MAX_TRANSFER_AMOUNT` (default ₹2,00,000), 2 decimals. Header `Idempotency-Key` (UUID) is mandatory. The same key from the same user returns the original result and never debits twice.
- **FR-T2 [M]** Atomic with `@Transactional`. Lock both account rows in ascending id order to avoid deadlocks. Insufficient funds, frozen account or exceeding the account's daily limit returns HTTP 422 with a clear message.
- **FR-T3 [M]** Every transfer goes through the fraud engine (section 5) before any money moves.
- **FR-T4 [M] Deposit (demo 'Add money')** credits the customer's own account, max `MAX_DEPOSIT_AMOUNT` (₹50,000) per operation. Type DEPOSIT. Not fraud-scored.

- **FR-T5 [M] History** with filters (type, status, date range, min/max amount, reference search) and pagination.
- **FR-T6 [M]** Reference format `TXN-` + 12 random uppercase alphanumerics, unique.

4.5 Notifications

- **FR-N1 [M]** In-app notifications for: transfer completed (sender and receiver), held for review, blocked, released after review, account frozen or unfrozen, password changed. Bell icon with unread count (poll every 30 s), mark one or all as read.
- **FR-N2 [S]** Email for held or blocked transfers through MailHog.

4.6 Admin

- **FR-AD1 [M]** Users: search, filter by role and status, create ANALYST or ADMIN, change role, disable or enable. An admin cannot demote or disable themselves.
- **FR-AD2 [M]** Accounts: list and freeze or unfreeze.
- **FR-AD3 [M]** Fraud rules: enable or disable, edit threshold, window and points (validated 0–100). Changes are audited.
- **FR-AD4 [M]** System settings (section 6) editable with validation.
- **FR-AD5 [M]** Audit log viewer with filters (actor, action, date range) and pagination. Read-only.
- **FR-AD6 [M]** Dashboard statistics: users, accounts, transaction count and volume, fraud alerts by status and by rule, transactions per day for the last 14 days.

4.7 Audit logging

Append-only `audit_logs` for: REGISTER, LOGIN_SUCCESS, LOGIN_FAILURE, LOGOUT, PASSWORD_CHANGED, TRANSFER_CREATED, TRANSFER_COMPLETED, TRANSFER_HELD, TRANSFER_BLOCKED, ALERT_RELEASED, ALERT_CONFIRMED_BLOCK, USER_ROLE_CHANGED, USER_STATUS_CHANGED, ACCOUNT_FROZEN, ACCOUNT_UNFROZEN, RULE_UPDATED, SETTING_UPDATED. There is no update or delete API for audit logs.

5. Fraud detection engine

Flow. Transfer request → validate → build `TransactionContext` (sender, account, beneficiary, amount, time, recent history) → run every enabled rule → sum points (cap 100) → decide → persist transaction (+ alert and hits if flagged) → move money only when COMPLETED → notify and audit.

Design. Strategy pattern. Interface `FraudRule { String code(); Optional<RuleHit> evaluate(TransactionContext ctx); }`. Each rule is a Spring `@Component`. `FraudEngine` receives `List<FraudRule>` by constructor injection. A new rule needs no change to `FraudEngine` (Open/Closed). Rule parameters (threshold, window, points, enabled) come from the `fraud_rules` table, so the Admin can tune them.

Code	Triggers when	Default points	Default parameters
R1 HIGH_AMOUNT	amount ≥ threshold	30	₹50,000
R2 VELOCITY	sender made ≥ N outgoing transfers in the window (including this one)	25	5 transfers, 10 min
R3 UNUSUAL_HOUR	server time between 00:00 and 04:59 in Asia/Kolkata	15	none
R4 NEW_BENEFICIARY_LARGE	beneficiary added less than H hours ago and amount ≥ threshold	30	24 h, ₹20,000
R5 AMOUNT_SPIKE	amount > M × sender's average completed outgoing transfer in 30 days (needs at least 3 earlier transfers)	20	M = 5
R6 DAILY_LIMIT_PROXIMITY	today's outgoing total including this one > P% of the account's daily limit	10	90%
R7 RECENT_FAILED_LOGINS	sender had ≥ N failed logins in the last W minutes	15	3, 30 min

Decision bands (from settings `REVIEW_THRESHOLD`=40 and `BLOCK_THRESHOLD`=70):

- Score < 40 → LOW → transaction `COMPLETED`, money moves.
- 40–69 → MEDIUM → `PENDING_REVIEW`, no money moves, alert OPEN (severity MEDIUM).
- ≥ 70 → HIGH → `BLOCKED`, no money moves, alert OPEN (severity HIGH).

Analyst resolution (FRAUD_ANALYST only, notes required):

- **Release** (false positive): re-check balance and account status, then execute the transfer in one transaction → `COMPLETED`, alert `RELEASED`. If funds are no longer available → `FAILED` with a reason.
- **Confirm block**: transaction stays or becomes `BLOCKED`, alert `CONFIRMED_BLOCK`.
- An alert is resolved once. A second attempt returns HTTP 409.

Explainability. Each flagged alert stores the rule hits (code, points, human-readable detail such as 'Amount ₹75,000 ≥ threshold ₹50,000'). Customers see only a generic message ('held for security review'). Analysts see the full breakdown.

Performance. Scoring must take under 200 ms on the seeded dataset. Use indexed queries only.

6. Data model (MySQL 9.7 LTS, schema owned by Flyway)

Enums are stored as `VARCHAR` with a `CHECK` constraint and mapped with `@Enumerated(EnumType.STRING)`. All timestamps are UTC (`DATETIME`). Keep the schema in 3NF.

Table	Columns and constraints
`users`	id PK; full_name; email UNIQUE; phone; password_hash; role (CUSTOMER, FRAUD_ANALYST, ADMIN); status (ACTIVE, DISABLED); failed_login_attempts INT; locked_until NULL; last_login_at NULL; created_at; updated_at
`accounts`	id PK; account_number UNIQUE; user_id FK; type (SAVINGS, CURRENT); balance DECIMAL(19,2) CHECK ≥ 0; daily_limit DECIMAL(19,2); status (ACTIVE, FROZEN, CLOSED); version BIGINT; created_at
`beneficiaries`	id PK; user_id FK; beneficiary_account_id FK; nickname; created_at; UNIQUE(user_id, beneficiary_account_id)

Table	Columns and constraints
`transactions`	id PK; reference UNIQUE; idempotency_key CHAR(36); initiated_by FK users; from_account_id FK NULL (deposit); to_account_id FK; type (TRANSFER, DEPOSIT); amount DECIMAL(19,2) CHECK > 0; description; status (COMPLETED, PENDING_REVIEW, BLOCKED, FAILED); risk_score INT NULL; failure_reason NULL; created_at; completed_at NULL; UNIQUE(initiated_by, idempotency_key); indexes on (from_account_id, created_at), (to_account_id, created_at), (status)
`fraud_rules`	id PK; code UNIQUE; name; description; threshold_value DECIMAL(19,2) NULL; window_minutes INT NULL; points INT CHECK 0..100; enabled BOOLEAN; updated_by FK NULL; updated_at
`fraud_alerts`	id PK; transaction_id FK UNIQUE; risk_score; severity (MEDIUM, HIGH); status (OPEN, RELEASED, CONFIRMED_BLOCK); reviewed_by FK NULL; review_notes NULL; created_at; reviewed_at NULL
`fraud_alert_hits`	id PK; alert_id FK (ON DELETE CASCADE); rule_code; points; detail
`refresh_tokens`	id PK; user_id FK; token_hash CHAR(64) UNIQUE (SHA-256, never store the raw token); expires_at; revoked BOOLEAN; created_at
`notifications`	id PK; user_id FK; title; message; is_read BOOLEAN; created_at; index (user_id, is_read)
`audit_logs`	id PK; actor_user_id NULL; action; entity_type; entity_id NULL; details; ip_address; created_at; index (action, created_at)
`system_settings`	setting_key PK; setting_value; updated_at

Settings keys and defaults: REVIEW_THRESHOLD 40, BLOCK_THRESHOLD 70, DEMO_OPENING_BALANCE 10000, MAX_TRANSFER_AMOUNT 200000, MAX_DEPOSIT_AMOUNT 50000, DEFAULT_DAILY_LIMIT 100000.

Migrations: `V1__create_users_and_accounts.sql`, `V2__create_beneficiaries_and_transactions.sql`, `V3__create_fraud_tables.sql`, `V4__create_auth_notifications_audit.sql`, `V5__create_system_settings.sql`, `V6__seed_fraud_rules_and_settings.sql`. Never edit an applied migration; add a new one. Also generate an ERD image into `docs/diagrams/`.

7. REST API (base path `/api/v1`, JSON, Bearer access token unless public)

Method and path	Who	Purpose
POST `/auth/register`	Public	Register a customer
POST `/auth/login`	Public	Login, returns access token, sets refresh cookie
POST `/auth/refresh`	Cookie	Rotate refresh token, return new access token
POST `/auth/logout`	Auth	Revoke refresh token
GET `/auth/me`	Auth	Current user
POST `/auth/change-password`	Auth	Change password [S]
GET `/public/stats`	Public	Aggregate, non-sensitive counters for the home page
GET, POST `/accounts`	CUSTOMER	List own accounts, open an account
GET `/accounts/{id}`	CUSTOMER (owner)	Account detail
GET `/accounts/{id}/statement`	CUSTOMER (owner)	Paginated statement; `.csv` export [S]
GET, POST `/beneficiaries`; DELETE `/beneficiaries/{id}`	CUSTOMER	Manage beneficiaries
GET `/beneficiaries/lookup?accountNumber=`	CUSTOMER	Verify an account, return masked holder name
POST `/transactions/transfer`	CUSTOMER	Transfer (requires `Idempotency-Key`)
POST `/transactions/deposit`	CUSTOMER	Demo add money

Method and path	Who	Purpose
GET `/transactions`; GET `/transactions/{reference}`	CUSTOMER	Own history and detail
GET `/notifications`; PATCH `/notifications/{id}/read`; PATCH `/notifications/read-all`	Auth	Notifications
GET `/fraud/alerts`; GET `/fraud/alerts/{id}`	ANALYST, ADMIN	Queue (filter by status, severity) and detail
POST `/fraud/alerts/{id}/release`; POST `/fraud/alerts/{id}/confirm-block`	ANALYST	Resolve (body: notes)
GET `/fraud/stats`	ANALYST, ADMIN	Open count, average score, resolved today, false-positive rate
GET, POST `/admin/users`; GET `/admin/users/{id}`; PATCH `/admin/users/{id}/role`; PATCH `/admin/users/{id}/status`	ADMIN	User management
GET `/admin/accounts`; PATCH `/admin/accounts/{id}/status`	ADMIN	Freeze or unfreeze
GET `/admin/transactions`	ADMIN	All transactions with filters
GET `/admin/fraud-rules`; PUT `/admin/fraud-rules/{id}`	ADMIN	Tune rules
GET, PUT `/admin/settings`	ADMIN	System settings
GET `/admin/audit-logs`	ADMIN	Audit viewer
GET `/admin/dashboard`	ADMIN	Statistics
GET `/actuator/health`	Public	Liveness (other actuator endpoints ADMIN only or disabled)

Conventions. Plural nouns; correct verbs and status codes (200, 201 with `Location`, 204, 400, 401, 403, 404, 409, 422, 429, 500). All list endpoints support `page`, `size`, `sort`. Never expose entities; use request and response DTOs (records). Errors use RFC 7807 `ProblemDetail` plus a `fieldErrors` list and a `requestId`. Document every endpoint in OpenAPI with examples and the bearer security scheme.

8. Security requirements

1. Passwords hashed with BCrypt (strength 12). Never log or return them.
2. Access JWT: HS256, 15 minutes, claims `sub` (user id), `role`, `iat`, `exp`, `jti`. Secret from `JWT_SECRET` (at least 32 random bytes); the app must refuse to start without it.
3. Refresh token: random 256-bit value, stored only as a SHA-256 hash, 7 days, rotated on every use, delivered in an HttpOnly cookie. The access token is kept in memory on the frontend (not in localStorage).
4. Stateless sessions (`SessionCreationPolicy.STATELESS`), CSRF handled deliberately: Bearer header for APIs, and `SameSite=Strict` on the refresh cookie, with the refresh endpoint accepting only POST.
5. RBAC: URL rules in `SecurityConfig` plus `@PreAuthorize` on service or controller methods. **Ownership checks in the service layer** on every account, beneficiary and transaction lookup (prevents IDOR). Return 404 (not 403) for other users' resources.
6. CORS limited to `FRONTEND_ORIGIN`, credentials allowed, no wildcard.
7. Rate limiting (Bucket4j): login 10 per minute per IP, transfer 20 per minute per user. Return 429 with `Retry-After`.
8. Validation on every request with Bean Validation. Reject unknown role values and negative or zero amounts.
9. Security headers: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, a strict `Content-Security-Policy` for the frontend (nginx).
10. Logging: SLF4J with a request id (MDC). Never log passwords, tokens, full account numbers or secrets. Mask account numbers (`TPG******1234`).
11. Generic login errors, constant-time password checks, no user enumeration on register-adjacent flows.
12. Map the implementation to OWASP Top 10 in `docs/SECURITY_AND_CONFIGURATION.md`.

9. Frontend requirements (React + TypeScript)

Look and feel. Modern, clean, fully responsive (mobile first), accessible (labels, focus states, keyboard navigation, AA contrast). Consistent design tokens in `styles/variables.css`. Skeleton loaders, empty states, toasts and confirmation modals everywhere they make sense.

Theme. Light and dark modes with a toggle in the navbar. Use Bootstrap `data-bs-theme` plus CSS variables. Default to the OS preference, persist the choice (`tpg-theme`), and avoid a flash of the wrong theme with a tiny inline script in `index.html`. Charts, tables, forms and modals must all respect the theme.

Home page (public, dynamic and engaging).

- Sticky navbar: logo, links (Features, How it works, Security, Videos, FAQ), theme toggle, Login and Register buttons.
- Hero with a muted, looping background video (`autoplay muted loop playsInline`, poster image, dark overlay for text contrast), animated headline (rotating words), and two call-to-action buttons.
- Animated counters (users, protected transactions, alerts reviewed) fed by `GET /public/stats`, with a graceful fallback when the API is down.
- 'How fraud detection works' section: animated step-by-step pipeline built with CSS or SVG (Transfer → Risk rules → Score → Approve, Hold or Block → Analyst review).
- Feature cards, security highlights, a video gallery (3 videos, click to open in a modal player with a title and description), a testimonial carousel (clearly labelled as demo content), FAQ accordion, final call to action, footer.
- Scroll-reveal animations using an `IntersectionObserver` hook. Respect `prefers-reduced-motion` (show the poster image instead of the video and disable animations).
- **Videos.** Files live in `frontend/public/media/`: `hero.mp4`, `hero-poster.jpg`, `how-it-works.mp4`, `security.mp4`, `fraud-demo.mp4` with matching posters. Each file at most 8 MB (compress with H.264). Add `frontend/public/media/README.md` explaining which video to record and where to put it (for example screen recordings of the app and royalty-free clips such as Pexels, with licence noted). If a video file is missing, show a gradient and poster fallback with no console errors. Do not generate or invent video content.

Public and auth pages. Login and Register (React Hook Form + Zod, show/hide password, password-strength meter, server errors mapped to fields), 403 and 404 pages.

Customer area (`/app/...`)

- Dashboard: greeting, total balance, account cards, quick actions (Transfer, Add money, Add beneficiary), income vs spending chart (last 6 months), last 5 transactions, notification bell, security card (last login, 'Fraud protection: Active').
- Accounts and statement with filters, pagination and CSV export [S].
- Transfer: choose account and beneficiary, amount, note, review-and-confirm step, then a result modal showing Completed, Held for review or Blocked with the reference. One idempotency key per form submission, reused on retry.
- Beneficiaries (lookup by account number, confirm masked name, add, delete).
- Transactions history, profile and settings (change password, theme), notifications page.

Analyst area (`/analyst/...`)

- Dashboard with KPIs (open alerts, average risk score, resolved today, false-positive rate).
- Alert queue (filters, severity badges, sorted by severity then age).
- Alert detail: transaction, customer summary, risk gauge, rule-hit breakdown, customer's recent history, required notes field, Release and Confirm block buttons with confirmation modals.

Admin area (`/admin/...`)

- Dashboard with KPIs and charts (transactions per day, alerts by rule).
- Users (search, filters, create, role change, disable or enable), accounts (freeze or unfreeze), fraud rules (toggle, edit threshold, window and points with validation), settings, audit logs, all transactions.

Routing and auth. `ProtectedRoute` (needs login) and `RoleRoute` (needs role). On refresh the app calls `/auth/refresh` once to restore the session. Axios interceptor: attach the access token; on 401 try one refresh and retry, otherwise log out; 403 → `/403`; 400 or 422 field errors → form fields; 429 → toast; 5xx → generic error toast. Redirect each role to its own home after login.

Frontend rules. No `any` types, API response types in `src/types`, one API module per backend feature, environment-based API URL (`VITE_API_BASE_URL`; Vite dev proxy for `/api`), `ErrorBoundary`, lazy-loaded route groups with `Suspense`.

10. Non-functional requirements

- Transfer API p95 under 500 ms on the seeded dataset. Pages load under 3 s on a normal connection.
- No N+1 queries on list endpoints (use join fetch or DTO projections). All list endpoints paginated.
- Backend line coverage at least 80% on `service`, `fraud` and `security` packages. SonarQube quality gate passes with no blocker or critical issues.
- Frontend works on the latest Chrome, Edge and Firefox, and from 360 px width upward.
- Time zone: store UTC, show Asia/Kolkata in the UI.
- Graceful errors: no stack traces to clients.

11. Repository structure

```
trustpay-guardian/
  README.md  LICENSE(MIT)  .gitignore  .gitattributes  .env.example
  docker-compose.yml  Jenkinsfile  sonar-project.properties  CONTRIBUTING.md
  .github/
    workflows/ci.yml
    pull_request_template.md
    ISSUE_TEMPLATE/bug_report.md  feature_request.md
  docs/
    PRD.md  DECISIONS.md  diagrams/ (erd.png, architecture.png, fraud-
flow.png)
    (14 generated documentation files in the final phase)
  database/
    README.md   (how to create the DB, run migrations, reset it)
  scripts/
    start-dev.sh  start-dev.ps1  reset-dev-db.sh
  backend/
    pom.xml  mvnw  mvnw.cmd  .mvn/  Dockerfile  .dockerignore
    src/main/java/com/trustpay/guardian/
      TrustPayGuardianApplication.java
      config/        OpenApiConfig  CorsConfig  AppProperties
SchedulingConfig  DevDataSeeder
      security/      SecurityConfig  JwtService  JwtAuthenticationFilter
AppUserDetailsService
                     RateLimitFilter  RestAuthenticationEntryPoint
RestAccessDeniedHandler  CurrentUser
      auth/          AuthController  AuthService  RefreshToken
RefreshTokenRepository  RefreshTokenService  dto/
      user/          User  Role  UserStatus  UserRepository  UserService
UserController  dto/
      account/       Account  AccountType  AccountStatus  AccountRepository
AccountService
                     AccountController  AccountNumberGenerator  dto/
      beneficiary/   Beneficiary  BeneficiaryRepository  BeneficiaryService
BeneficiaryController  dto/
      transaction/   Transaction  TransactionType  TransactionStatus
TransactionRepository
                     TransferService  TransactionQueryService
TransactionController  dto/
      fraud/
        engine/      FraudRule(interface)  FraudEngine  TransactionContext
RuleHit  RiskDecision
        rules/       HighAmountRule  VelocityRule  UnusualHourRule
NewBeneficiaryLargeRule
                     AmountSpikeRule  DailyLimitProximityRule
```

```
RecentFailedLoginsRule
        config/      FraudRuleConfig(entity)  FraudRuleConfigRepository
FraudRuleConfigService
        alert/       FraudAlert  FraudAlertHit  FraudAlertRepository
FraudAlertService
                     FraudAlertController  FraudStatsService  dto/
      notification/  Notification  NotificationRepository
NotificationService  NotificationController
      audit/         AuditLog  AuditLogRepository  AuditService
AuditController
      admin/         AdminUserController  AdminAccountController
AdminRuleController
                     AdminSettingsController  AdminDashboardController
SystemSetting  SettingsService  dto/
      publicapi/     PublicStatsController
      common/        exception/ (GlobalExceptionHandler, NotFoundException,
ConflictException, BusinessRuleException)
                     web/ (RequestIdFilter, PageResponse)  util/
(MaskingUtils, MoneyUtils)
    src/main/resources/  application.yml  application-dev.yml  application-
prod.yml
                         logback-spring.xml  db/migration/V1..V6 (.sql)
    src/test/java/...     (mirrors main; unit, web-layer and integration
tests)
    src/test/resources/   application-test.yml
  frontend/
    package.json  vite.config.ts  tsconfig.json  index.html  Dockerfile
nginx.conf  .env.example
    playwright.config.ts  eslint and prettier config
    public/  favicon.svg  media/ (hero.mp4, posters, other videos, README.md)
    src/
      main.tsx  App.tsx
      routes/      AppRoutes  ProtectedRoute  RoleRoute
      api/         axiosClient  authApi  accountApi  beneficiaryApi
transactionApi
                   notificationApi  fraudApi  adminApi  publicApi
      context/     AuthContext  ThemeContext
      store/       store  notificationsSlice  alertsSlice
      hooks/       useCountUp  useInView  useDebounce  usePolling
      components/  layout/ common/ home/ charts/ forms/
      pages/       home/ auth/ customer/ analyst/ admin/ errors/
      types/  utils/ (formatCurrency, formatDate, mask)  styles/
(variables.css, theme.css, custom.css)
      tests/
```

12. Configuration (all through environment variables; list them in `.env.example` with safe placeholders)

`SPRING_PROFILES_ACTIVE`, `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `JWT_SECRET`, `JWT_ACCESS_TTL_MINUTES` (15), `JWT_REFRESH_TTL_DAYS` (7), `FRONTEND_ORIGIN`, `COOKIE_SECURE`, `MAIL_HOST`, `MAIL_PORT`, `DEV_SEED_PASSWORD` (dev only), `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, `VITE_API_BASE_URL`. Profiles: `dev` (verbose logs, seeder on, Swagger on), `test` (Testcontainers), `prod` (seeder off, Swagger restricted or off, secure cookies).

13. Git and GitHub requirements

- Branches: `main` (always working), `develop`, and `feature/<phase>-<short-name>`. Merge through pull requests.
- Conventional Commits: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`, `ci:`. One logical change per commit; commit at the end of every phase. Tag milestones (`v0.1.0` after the backend, `v1.0.0` at release).
- `.gitignore` must cover `target/`, `node_modules/`, `dist/`, `.env`, `*.log`, IDE folders and OS files. Only `.env.example` is committed.
- Provide a pull request template, issue templates, an MIT `LICENSE`, `CONTRIBUTING.md` and a README with: title, badges (build status), screenshots, features, architecture diagram, tech stack, prerequisites, quick start (Docker Compose), manual setup, environment variables, demo accounts (dev only), API docs link, testing commands, folder structure, roadmap and links to `docs/`.
- Keep large binaries out of Git. Videos stay under 8 MB each or use Git LFS.

14. Testing and quality

- **Unit (JUnit 5 + Mockito):** every fraud rule with boundary values, `FraudEngine` scoring and bands, `TransferService` (success, insufficient funds, frozen, idempotent replay, flagged), `JwtService`, lockout logic, `RefreshTokenService` rotation and reuse detection.
- **Web layer (MockMvc):** status codes, validation errors, 401 versus 403, ownership checks (user A cannot read user B's account).
- **Integration (`@SpringBootTest` + Testcontainers MySQL):** register → login → transfer → balance check; concurrent transfers keep balances correct; flagged transfer → analyst release.
- **Frontend:** component tests (login form, transfer form, ProtectedRoute, theme toggle) and one Playwright smoke test (home → login → dashboard).

- **Tools:** JaCoCo report (target ⩾ 80% on key packages), SonarQube scan, ESLint, TypeScript strict mode.

15. Docker, CI/CD and deployment

- `backend/Dockerfile` (multi-stage: Maven build, then a slim Java 25 runtime (Eclipse Temurin 25 JRE), non-root user). `frontend/Dockerfile` (Node build, then nginx serving `dist` and proxying `/api` to the backend).
- `docker-compose.yml`: `mysql` (9.7 LTS, volume, healthcheck), `backend` (depends on healthy mysql), `frontend`, `mailhog`. One command starts everything: `docker compose up --build`.
- `Jenkinsfile` stages: Checkout → Build → Test → SonarQube scan → Docker build → (optional) Deploy. `.github/workflows/ci.yml`: run backend tests and frontend lint, test and build on every pull request.
- Production notes in `docs/DEPLOYMENT_GUIDE.md`: environment variables, secrets handling, HTTPS, health checks, backups.

16. Seed data and demo script (dev profile only)

The `DevDataSeeder` creates, using `DEV_SEED_PASSWORD` (never a hard-coded password): 1 ADMIN (`admin@trustpay.test`), 1 FRAUD_ANALYST (`analyst@trustpay.test`), 3 CUSTOMERs (`asha@trustpay.test`, `ravi@trustpay.test`, `meera@trustpay.test`) with accounts, balances, beneficiaries (one added less than 24 hours ago), 30 days of normal transactions, and the default fraud rules and settings.

Demo script (document in the README and test in the integration suite): a normal ₹500 transfer completes (score 0); a ₹60,000 transfer to a new beneficiary scores 60 and is held (R1 + R4); several rapid large transfers push the score to 70 or more and are blocked; the analyst releases one and confirms another; the admin lowers a rule's points and the next transfer scores differently.

17. Syllabus coverage and beyond-syllabus items

Covered from the Java Full Stack syllabus: Core Java and OOP, collections, exceptions, streams, records, Optional, java.time; Git branching; Maven; SQL (joins, aggregation, indexes, constraints, transactions); Spring Core and Boot; REST design, DTOs, validation, `@ControllerAdvice`, OpenAPI; Spring Data JPA with pagination and `@Transactional`; Spring Security with JWT and refresh tokens; JUnit 5, Mockito, MockMvc, Testcontainers, JaCoCo, SonarQube; SLF4J/Logback and Actuator; SOLID and patterns (Repository, Strategy, Builder, Factory); HTML5, CSS3, Bootstrap 5; JavaScript, TypeScript; React

(hooks, Router, Context, Redux Toolkit, Axios, React Hook Form, Zod); frontend testing; CORS, `.env` and end-to-end flow; Docker and Jenkins; OWASP and secure coding; code review and Agile practices.

Beyond the syllabus (added for project complexity): Java 25 (the syllabus stops at 21), MySQL 9.7 LTS, React Router v7, Flyway migrations, idempotency keys, pessimistic row locking with ordered locks, refresh-token rotation with reuse detection and HttpOnly cookies, rate limiting (Bucket4j), append-only audit trail, Strategy-based configurable fraud engine, MailHog, Vitest (replaces Jest for Vite projects), Recharts, GitHub Actions.

18. Definition of done

1. `docker compose up --build` starts everything and the home page loads.
2. Register, login, refresh, logout all work; lockout works after 5 failures.
3. Each role sees only its own area; direct URL access to another role's pages and APIs returns 403.
4. A user cannot access another user's account or transaction (404).
5. Transfers are atomic and idempotent; concurrent transfers never overdraw an account.
6. The demo script in section 16 behaves exactly as described.
7. Analyst release and confirm-block both work and notify the customer.
8. Admin can manage users, accounts, rules, settings and view audit logs.
9. Light and dark themes work on every page; the layout works at 360 px width.
10. All backend and frontend tests pass; coverage and SonarQube targets are met.
11. Swagger lists every endpoint with examples; `.env.example` and README are complete.
12. No secrets, `node_modules` or `target` folders are in Git; the 14 documentation files exist and are cross-linked.